

SystemVerilog for Verilog Users: Quick Reference

The processor code in this project is written in **SystemVerilog** (.sv files), not plain Verilog. SystemVerilog is a superset of Verilog — all Verilog is valid SystemVerilog, but a few new keywords are used in this codebase. This section explains everything you need to know to read and modify the provided files.

1. logic replaces wire and reg

In Verilog, you had to decide whether a signal is `wire` or `reg`, and getting it wrong caused errors. SystemVerilog introduces a single type, **logic**, that works in both contexts.

Verilog	SystemVerilog
<code>wire [31:0] data;</code>	<code>logic [31:0] data;</code>
<code>reg [31:0] count;</code>	<code>logic [31:0] count;</code>
<code>output reg [31:0] y;</code>	<code>output logic [31:0] y;</code>
<code>input wire [31:0] a;</code>	<code>input logic [31:0] a;</code>

Rule of thumb: Everywhere you would write `wire` or `reg`, write `logic` instead.

2. always_comb replaces always @(*)

For **combinational logic**, SystemVerilog uses `always_comb` instead of `always @(*)`. They behave the same way, but `always_comb` tells the synthesizer you *intend* combinational logic and it will warn you if you accidentally create a latch.

// Verilog	// SystemVerilog
<code>always @(*)</code>	<code>always_comb</code>
<code>case(op)</code>	<code>case(op)</code>
<code>6'b000000: ctrl = 9'b110;</code>	<code>6'b000000: ctrl = 9'b110;</code>
<code>default: ctrl = 9'bxxx;</code>	<code>default: ctrl = 9'bxxx;</code>
<code>endcase</code>	<code>endcase</code>

Inside `always_comb`, use **blocking assignment** (`=`). Note: the provided code uses `<=` inside `always_comb` in some places — this works in simulation but `=` is the proper style.

3. always_ff replaces always @(posedge clk)

For **sequential logic** (flip-flops), SystemVerilog uses `always_ff`:

// Verilog	// SystemVerilog
<code>always @(posedge clk)</code>	<code>always_ff @(posedge clk)</code>
<code>if (we)</code>	<code>if (we)</code>
<code>rf[wa] <= wd;</code>	<code>rf[wa] <= wd;</code>

Inside `always_ff`, use **non-blocking assignment** (`<=`), just like you do in Verilog.

If you see `always_ff @(posedge clk, posedge reset)`, it is the same as the Verilog `always @(posedge clk or posedge reset)` — the comma simply replaces `or` in the sensitivity list.

4. ANSI-style module ports (types declared inline)

In Verilog-2001+, you can already declare types inside the port list. The project uses this style throughout:

```
module alu(input logic [31:0] a, b,
          input logic [2:0] f,
          output logic [31:0] y,
          output logic zero);
```

This is equivalent to the older Verilog style:

```

module alu(a, b, f, y, zero);
  input  [31:0] a, b;
  input  [2:0] f;
  output [31:0] y;
  output      zero;
  reg    [31:0] y;
  reg      zero;

```

5. Named port connections with .name(signal)

When instantiating modules, you can connect ports **by name** instead of by position:

```

// By position (Verilog style – used for most modules in the project):
adder pcadd1(pc, 32'b100, pcplus4);

// By name (used for the ALU):
alu alu(.a(srca), .b(srcb), .f(alucontrol),
        .y(aluout), .zero(zero));

```

The `.portname(signalname)` syntax makes it clear which port connects to which signal. Order does not matter when using named connections. Both styles can coexist in the same design — use whichever is clearer.

6. === (case equality) in testbenches

In the testbench, you will see `===` instead of `==`. The difference:

- `==` returns **x** (unknown) if either operand contains x or z bits
- `===` returns **true/false** even when comparing x values

This matters only in simulation (testbenches), not in synthesizable code.

Summary Cheat Sheet

Verilog	SystemVerilog	Where used
wire/reg	logic	Everywhere
always @(*)	always_comb	Combinational blocks
always @(posedge clk or posedge reset)	always_ff @(posedge clk, posedge reset)	Sequential blocks
Port types declared separately	Port types declared inline	Module headers
.port(signal)	.port(signal)	Same — both work in Verilog-2001+

That's it. These six points cover every SystemVerilog feature used in this project. If your simulator supports `.sv` files (ModelSim, Vivado, Icarus Verilog 12+), no further changes are needed.